Ex-6: Gradient descent

Problem 1:

Aim:

To implement linear regression using Gradient Descent in Python in order to find the optimal slope and intercept that minimize the error between predicted and actual values.

Coding:

colab.research.google.com/drive/1tqy5mkHbP90rCQMsvKq6m_6-F3Y4Hg5

[1]
0s

```python
import numpy as np

def gradient_descent(x, y, lr=0.01, epochs=100):
    m, c = 0, 0
    n = len(x)
    for _ in range(epochs):
        y_pred = m*x + c
        dm = (-2/n) * sum(x*(y - y_pred))
        dc = (-2/n) * sum(y - y_pred)
        m -= lr * dm
        c -= lr * dc
    return m, c

x = np.array([1,2,3,4,5])
y = np.array([2,4,6,8,10])
m, c = gradient_descent(x, y)
print('Slope:', m, 'Intercept:', c)
```

```
Slope: 1.8984326022295357 Intercept: 0.36669053309859806
```

Output:

Slope:1.8984326022295357 **Intercepts:**0.36669053309859806

Problem 2:

Aim:

To implement linear regression using Gradient Descent in Python in order to find the optimal slope and intercept that minimize the error between predicted and actual values.

Codinig:

colab.research.google.com/drive/1tqy5mkHbP90rCQMsvKq6m_6-F3Y4Hg5

[2]
0s

```
import numpy as np

def gradient_descent(x, y, lr=0.01, epochs=100):
    m, c = 0, 0
    n = len(x)
    for _ in range(epochs):
        y_pred = m*x + c
        dm = (-2/n) * sum(x*(y - y_pred))
        dc = (-2/n) * sum(y - y_pred)
        m -= lr * dm
        c -= lr * dc
    return m, c

x = np.array([2,3,4,5,6])
y = np.array([2,4,5,7,10])
m, c = gradient_descent(x, y)
print('Slope:', m, 'Intercept:', c)
```

```
Slope: 1.482367679166547 Intercept: -0.13170645285831592
```

Output:

Slope:1.4823679166547 **Intercepts:** -0.1317064528531592

Problem 3:

Aim:

To implement linear regression using Gradient Descent in Python in order to find the optimal slope and intercept that minimize the error between predicted and actual values.

Codinig:

colab.research.google.com/drive/1tqy5mkHbP90rCQMsvKq6m_6-F3Y4Hg5

[3]
0s

```python
import numpy as np

def gradient_descent(x, y, lr=0.01, epochs=100):
    m, c = 0, 0
    n = len(x)
    for _ in range(epochs):
        y_pred = m*x + c
        dm = (-2/n) * sum(x*(y - y_pred))
        dc = (-2/n) * sum(y - y_pred)
        m -= lr * dm
        c -= lr * dc
    return m, c

x = np.array([2,3,4,5,6])
y = np.array([2,3,6,8,10])
m, c = gradient_descent(x, y)
print('Slope:', m, 'Intercept:', c)
```

```
Slope: 1.571323664749143 Intercept: -0.23494697032899925
```

Output:

Slope:1.571323664749143 **Intercepts:** -0.23494697032899925

Problem 4:

Aim:

To implement linear regression using Gradient Descent in Python in order to find the optimal slope and intercept that minimize the error between predicted and actual values.

Codinig:

colab.research.google.com/drive/1tqy5mkHbP90rCQMsvKq6m_6-F3Y4Hg5

+ Code + Text

[4]
0s

```python
import numpy as np

def gradient_descent(x, y, lr=0.01, epochs=100):
    m, c = 0, 0
    n = len(x)
    for _ in range(epochs):
        y_pred = m*x + c
        dm = (-2/n) * sum(x*(y - y_pred))
        dc = (-2/n) * sum(y - y_pred)
        m -= lr * dm
        c -= lr * dc
    return m, c

x = np.array([3,4,5,6,7])
y = np.array([2,3,6,8,10])
m, c = gradient_descent(x, y)
print('Slope:', m, 'Intercept:', c)
```

```
Slope: 1.3084021160143229 Intercept: -0.4367089641272674
```

Output:

Slope:1.3084021160143229 **Intercepts:** -0.4367089641272674

Problem 5:

Aim:

To implement linear regression using Gradient Descent in Python in order to find the optimal slope and intercept that minimize the error between predicted and actual values.

Codinig:

```
[1]
✓ 0s
import numpy as np

def gradient_descent(x, y, lr=0.01, epochs=100):
    m, c = 0, 0
    n = len(x)

    for _ in range(epochs):
        y_pred = m * x + c

        dm = (-2/n) * sum(x * (y - y_pred))
        dc = (-2/n) * sum(y - y_pred)

        m -= lr * dm
        c -= lr * dc

    return m, c

x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 5, 4, 5])

m, c = gradient_descent(x, y)

print("Slope:", m, "Intercept:", c)

Slope: 0.9728893670773188 Intercept: 0.8537510676872087
```

Output:

Slope: 0.9728893670773188 **Intercept:** 0.8537510676872087

Problem 6:

Aim:

To implement linear regression using Gradient Descent in Python in order to find the optimal slope and intercept that minimize the error between predicted and actual values.

Coding:

```
[2]
0s
import numpy as np

def gradient_descent(x, y, lr=0.01, epochs=100):
    m, c = 0, 0
    n = len(x)

    for _ in range(epochs):
        y_pred = m * x + c

        dm = (-2/n) * sum(x * (y - y_pred))
        dc = (-2/n) * sum(y - y_pred)

        m -= lr * dm
        c -= lr * dc

    return m, c


# INPUT SET 1
x = np.array([2, 4, 6, 8, 10])
y = np.array([3, 5, 7, 9, 11])

m, c = gradient_descent(x, y)

print("Slope:", m, "Intercept:", c)

Slope: 1.0811224471996035 Intercept: 0.40751525352730406
```

Output:

Slope: 1.0811224471996035 **Intercept:** 0.40751525352730406

Problem 7:

Aim:

To implement linear regression using Gradient Descent in Python in order to find the optimal slope and intercept that minimize the error between predicted and actual values.

Coding:

```
[4]
0s
import numpy as np

def gradient_descent(x, y, lr=0.01, epochs=100):
    m, c = 0, 0
    n = len(x)

    for _ in range(epochs):
        y_pred = m * x + c

        dm = (-2/n) * sum(x * (y - y_pred))
        dc = (-2/n) * sum(y - y_pred)

        m -= lr * dm
        c -= lr * dc

    return m, c


x = np.array([1, 3, 5, 7, 9])
y = np.array([2, 6, 10, 14, 18])

m, c = gradient_descent(x, y)

print('Slope:', m, 'Intercept:', c)

Slope: 1.971692265295127 Intercept: 0.1854895716282358
```

Output:

Slope: 1.971692265295127 **Intercept:** 0.1854895716282358